Billions of spin flips per second from Python: a roofline study of GPU Ising Monte Carlo

Walid Hemati
Independent researcher

Abstract

We implement checkerboard Metropolis and heat-bath Monte Carlo for the two- and three-dimensional Ising model in seven Python-driven backends (NumPy, PyTorch eager, `torch.compile`, Triton, NVIDIA Warp, Numba-CUDA and a hand-written CUDA C reference compiled at run time through CuPy) and measure every rung of the optimisation ladder against a roofline built from two measured ceilings: device-to-device copy bandwidth (360.9 GB/s) and the throughput of the Philox4x32-10 counter-based generator (112.6 calls/ns). All backends share one counter-based random stream, so the same seed produces bit-identical spin configurations on every backend (33 of 33 identity checks pass). With 64 spins per 64-bit word, bit-sliced neighbour counting, and a *lazy bitwise comparison* that draws on average 7.6 random bits per spin instead of 32 while sampling the Metropolis acceptance probability exactly, the fastest variant (CUDA C (CuPy) (msc)) reaches **419.3 spin flips per ns** on a single NVIDIA GeForce RTX 3070 Laptop GPU, 43.6 % of the measured copy bandwidth under a minimal 0.375-byte-per-update traffic model. The fastest pure-Python-DSL kernel (Warp (msc)) reaches 371.9 flips/ns. Measured energies agree with the exact finite-lattice solution (Kaufman, Ferdinand–Fisher) to within 3.29 standard errors at every temperature, magnetisations with Onsager–Yang, and Binder-cumulant crossings give T_c to 0.09 % (2D) and 0.03 % (3D).

1 Introduction

The Ising model is the standard benchmark of Monte Carlo simulation on new hardware. Its update is a handful of integer operations per spin, so on a GPU the cost is set by memory traffic and by the random number generator, not by floating-point arithmetic. This makes it a clean test of a broader claim: that Python, used as a host for GPU kernels, can reach the same *hardware efficiency* as hand-tuned CUDA. We test that claim with a roofline [1]: every throughput below is quoted as an absolute rate and as a fraction of a ceiling measured on the same machine.

Contributions:

- An open-source package, `pyising`, with one common interface over seven backends and fourteen backend/variant combinations in 2D and 3D, Metropolis and heat bath, plus a Wolff cluster sampler [2] for near-critical validation.
- A layout-independent Philox4x32-10 [3] random stream and integer acceptance thresholds, so that the same seed gives bit-identical trajectories on NumPy, PyTorch, Triton, Warp, Numba and CUDA C. The NumPy implementation is the test oracle.
- A lazy, lane-parallel bitwise comparison of uniforms against the Metropolis thresholds for 1-bit multi-spin coding. It is exact (equivalent to a 32-bit integer comparison) and needs on average 3.80 Philox calls per 64-spin word at T_c (6.30 counting SIMT divergence) instead of 16.

- A roofline analysis with two measured ceilings (copy bandwidth and Philox throughput), a minimal traffic model for int8 and packed spins, and an honest comparison with published GPU Ising codes.
- Validation against exact results: the finite-torus energy at all temperatures, the spontaneous magnetisation, Binder crossings in 2D and 3D.

2 Background and related work

Model. We simulate $H = -J \sum_{\langle ij \rangle} s_i s_j$ with $s_i = \pm 1$, $J = k_B = 1$, periodic boundaries, on $L \times L$ square and L^3 simple-cubic lattices. In 2D the critical temperature is $T_c = 2/\ln(1+\sqrt{2}) \approx 2.269185$ [4], the spontaneous magnetisation is $m_0 = (1 - \sinh^{-4}(2/T))^{1/8}$ [5], and the free energy of a finite torus is known in closed form [6]. In 3D the most precise Monte Carlo estimate is $K_c = J/k_B T_c = 0.221\,654\,626(5)$ [7].

Algorithms. Metropolis [8] and heat-bath updates of one sublattice of a checkerboard decomposition are independent, which makes them data-parallel. Near T_c local updates slow down critically; cluster algorithms [2, 9] avoid this and are used here only for validation. The Binder cumulant $U_4 = 1 - \langle m^4 \rangle / (3\langle m^2 \rangle^2)$ [10] is size-independent at T_c up to corrections to scaling.

GPU Ising codes. Preis et al. [11] ported checkerboard Metropolis to a GeForce GTX 280 and reported speedups of about 60 (2D) and 35 (3D) over one CPU core. Block et al. [12] used multi-spin coding with one bit per spin and reached 7.9 flips/ns on one Tesla C1060 and 206 flips/ns on 64 such GPUs for an $800\,000^2$ lattice. Weigel [13] reported about 0.1 ns per spin flip in 2D on a Tesla C1060, and Weigel [14] studied the performance potential in detail (e.g. 0.27 ns per flip without multi-hit updates on the C1060). Yang et al. [15] ran the model on TPU clusters (12.9 flips/ns per TPUv3 core, 5853 flips/ns on 512 cores). Romero et al. [16] gave the reference implementations for current NVIDIA hardware: on one Tesla V100-SXM their basic Python version (Numba kernels plus CuPy random numbers) reaches 43.8 flips/ns, their basic CUDA C version 67.0 flips/ns, and their optimised multi-spin-coding version (4 bits per spin, 16 spins per 64-bit word, cuRAND Philox4_32_10 random numbers) 420.8 flips/ns; 16 V100s in a DGX-2H reach 7381 flips/ns. Our contribution is not a larger aggregate number but the measured efficiency of Python-hosted kernels against the ceilings of the machine they run on.

3 Method

3.1 Update rules with integer thresholds

For a site with spin s and neighbour sum h, Metropolis flips with probability $\min(1, e^{-2\beta sh})$ and heat bath sets $s' = +1$ with probability $1/(1 + e^{-2\beta h})$. Both depend only on $X = sh$ (resp. h) $\in \{-2d, \ldots, 2d\}$, so we precompute a table of $2d + 1$ unsigned 32-bit thresholds $t_X = \lfloor p_X 2^{32} \rfloor$ and accept iff $u < t_X$ for a uniform 32-bit word u. The comparison is exact integer arithmetic, so every backend takes the same decision for the same u; floating-point differences between compilers cannot change a trajectory.

3.2 A layout-independent random stream

Philox4x32-10 [3] maps a 128-bit counter and a 64-bit key to four 32-bit words. Sites of colour c are numbered $n = r\,(L_x/2) + j$ in the compressed checkerboard layout (r the flattened row, j the position among the colour-c sites of the row). Site n in sweep t uses word $n \bmod 4$ of $\mathrm{Philox}(\lfloor n/4 \rfloor, t, c, 0; \mathrm{seed})$, so one call serves four sites and the number does not depend on which thread computes it. Our implementation reproduces the Random123 known-answer vectors.

3.3 Multi-spin coding with lazy bitwise uniforms

In the packed variant bit b of word w holds the spin at $j = 64w + b$ (1 = up). The anti-aligned indicators $x_k = s \oplus n_k$ for the four neighbour words (the horizontal one is a one-bit funnel shift with a carry from the adjacent word) are reduced bit-sliced: $a_0 = \neg(x_1 \vee x_2 \vee x_3 \vee x_4)$, $a_{\geq 2} = (x_1 \wedge x_2) \vee (x_3 \wedge x_4) \vee ((x_1 \oplus x_2) \wedge (x_3 \oplus x_4))$ and $a_1 = \neg a_0 \wedge \neg a_{\geq 2}$, where a is the number of anti-aligned neighbours and $\Delta E = 8 - 4a$. Lanes with $a \geq 2$ always flip; lanes with $a = 1$ ($a = 0$) flip iff their uniform is below t_4 (t_8).

Instead of drawing a 32-bit uniform for every lane we build all 64 uniforms at once, most significant bit first: random word R_k supplies bit $31 - k$ of every lane's u. With T_k the lane mask of the threshold bits (t_4 on a_1 lanes, t_8 on a_0 lanes) and D the mask of still-undecided lanes,

$$\mathrm{acc} \mid= D \wedge \neg R_k \wedge T_k, \qquad D \;\&= \neg(R_k \oplus T_k). \tag{1}$$

A lane is decided at the first bit where u and the threshold differ, exactly as in an integer comparison; if all 32 bits agree, $u = t$ and the move is rejected. The acceptance probability is therefore exactly $t/2^{32}$, identical to the int8 kernels. The loop stops when $D = 0$; each lane survives a level with probability 1/2, so a word needs about $\log_2 n + 1.3$ levels for n lanes that need a random decision, and two levels come from one Philox call (128 bits). We measured 3.80 Philox calls per word at T_c, i.e. 7.6 random bits per spin instead of 32. On SIMT hardware a warp runs the loop until its slowest thread is done; the per-warp maximum is 6.30 calls per word (12.6 bits per spin), still 2.5× fewer than 16. Since results do not depend on where the loop stops, an eager ablation that always draws 32 levels produces bit-identical configurations and isolates the cost of randomness.

3.4 Measurements and errors

Energy and magnetisation are reduced on the device (every bond has exactly one colour-0 end, so $E = -\sum_{i \in 0} s_i h_i$; packed lattices use population counts). Errors come from 32-block binning; Binder errors from a blocked jackknife. Validation runs start from Wolff-equilibrated configurations, so critical slowing down of the local dynamics affects only the error bars, not the mean.

4 Implementation: the optimisation ladder

Table 1 lists the rungs. All GPU kernels use the compressed checkerboard layout: one array per colour of shape $(\mathrm{rows}, L_x/2)$, so a half sweep reads the opposite colour array, reads and writes its own, and touches no inactive sites.

Memory-traffic model. Per half sweep of $N/2$ updates the minimal DRAM traffic is one read of the own colour, one read of the other colour (neighbours shared through caches) and one write of the own colour: 3× the spin size per update, i.e. 3 B for int8 and 0.375 B for 1-bit packing. The copy-bandwidth ceiling is then 120 flips/ns for int8 and 962 flips/ns for packed spins. The Philox

rung	backend	what changes
0	NumPy	reference: vectorised Philox on `uint64`, compressed layout
1	PyTorch eager	same algorithm as tensor ops (dozens of kernels per half sweep)
2	`torch.compile`	the same Python, fused by Inductor into Triton kernels
3	Triton	hand-written kernel, $[B, 4]$ tiles, one Philox call per 4 sites
4	Warp	kernel in Python, 3D launch grid without integer division
5	Numba-CUDA	kernel in Python, compile-time constants by closure
6	CUDA C (CuPy)	`char4` loads, one thread per 4 sites, NVRTC
7	all GPU DSLs	1-bit multi-spin coding with lazy bitwise uniforms

Table 1: Backends and variants. Every row is a registry entry with the same interface; rows 1–6 reproduce rung 0 bit for bit, rung 7 reproduces its own NumPy reference bit for bit.

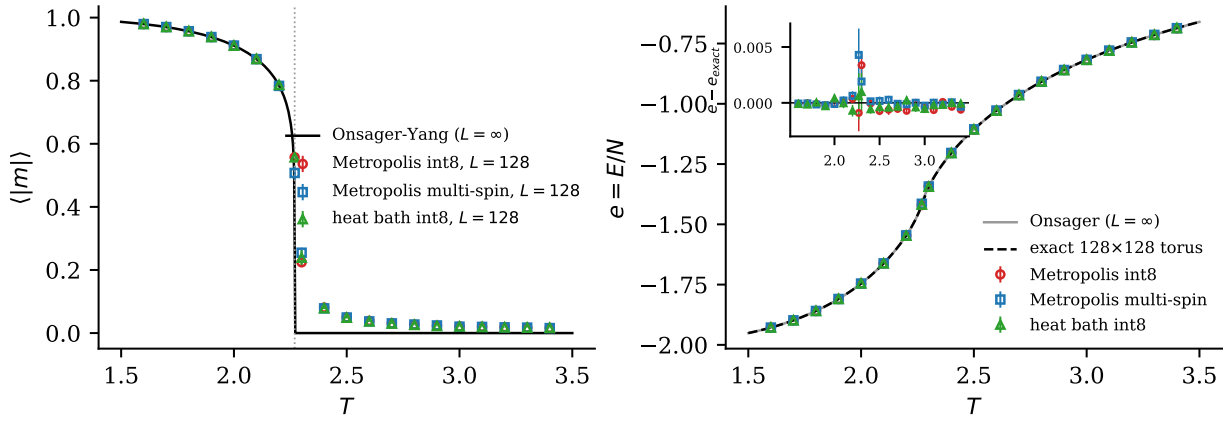


Figure 1: 2D validation at $L = 128$. Left: $\langle|m|\rangle$ against the Onsager–Yang spontaneous magnetisation. Right: energy per spin against Onsager's infinite-lattice result and the exact energy of the finite torus; inset: deviation from the exact finite-lattice value.

ceiling is 450 flips/ns for one call per four sites and 1144 flips/ns for the lazy packed kernel when the per-warp number of calls is charged (1894 flips/ns with the per-thread mean).

5 Validation

Bitwise identity. With the same seed and start, all of the 33 backend/variant/algorithm/dimension combinations reproduce the NumPy oracle bit for bit (table 1); the test suite checks this on every run.

Exact results. Figure 1 shows Metropolis (int8 and multi-spin) and heat bath at $L = 128$. Against the exact finite-torus energy the largest deviation is 4.3×10^{-3} and the largest deviation in units of the standard error is 3.29. Below T_c, away from the critical region, $\langle|m|\rangle$ agrees with the infinite-lattice Onsager–Yang result to 3.8×10^{-4}.

Critical temperature. The crossing of the two largest 2D Binder curves (fig. 2) is at $T = 2.2712$ (0.09 % from Onsager's T_c); in 3D at $T = 4.5102$ (0.03 % from $1/K_c$ of 7), with the residual shift

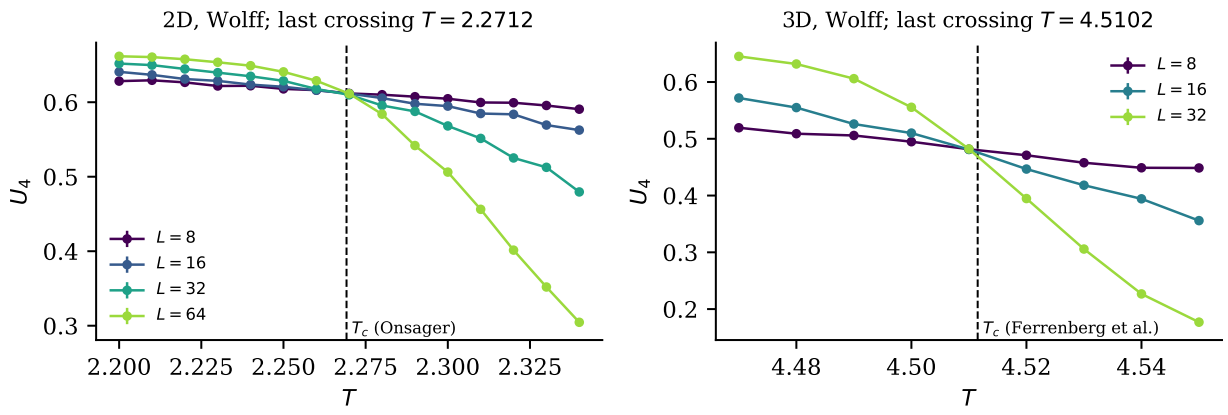


Figure 2: Binder cumulants from Wolff simulations; the dashed line is the exact (2D) or reference (3D, 7) critical temperature.

backend	variant	L	flips/ns	GB/s	% copy
NumPy	int8	1024	0.05	0.1	0.0
torch eager	int8	8192	0.49	1.5	0.4
torch.compile	int8	8192	9.60	28.8	8.0
Triton	int8	2048	58.33	175.0	48.5
Warp	int8	2048	45.31	135.9	37.7
Numba-CUDA	int8	2048	43.64	130.9	36.3
CUDA C (CuPy)	int8	2048	80.81	242.4	67.2
CUDA C (CuPy)	int8-rng-per-site	4096	49.91	149.7	41.5
NumPy	msc	1024	0.12	0.0	0.0
Triton	msc	131072	342.43	128.4	35.6
Warp	msc	16384	371.93	139.5	38.6
Numba-CUDA	msc	16384	327.36	122.8	34.0
CUDA C (CuPy)	msc	4096	419.32	157.2	43.6
CUDA C (CuPy)	msc-eager32	4096	211.81	79.4	22.0

Table 2: Best median throughput per backend and variant (2D Metropolis at T_c) with effective bandwidth under the minimal traffic model. Source: `results/rtx3070-laptop.json`.

expected from corrections to scaling at these sizes. 3D Metropolis and heat bath on the GPU agree with 3D Wolff energies at $L = 32$ within 1.19 standard errors.

6 Performance

Measurements use `bench/run.py`: warm-up including JIT compilation, device synchronisation around the timed region, at least 0.5 s per timed run and the median of five repeats. Figures 3 and 4 and table 2 summarise 98 runs on the NVIDIA GeForce RTX 3070 Laptop GPU (copy bandwidth 360.9 GB/s measured, 448 GB/s nominal).

int8 rungs. NumPy reaches 0.049 flips/ns. The tensor-op versions are dominated by temporaries (PyTorch eager) or by the 64-bit emulation of Philox (`torch.compile`). The best int8 kernel (CUDA C (CuPy)) reaches 80.8 flips/ns, 67.2 % of the copy-bandwidth ceiling: int8 Metropolis is

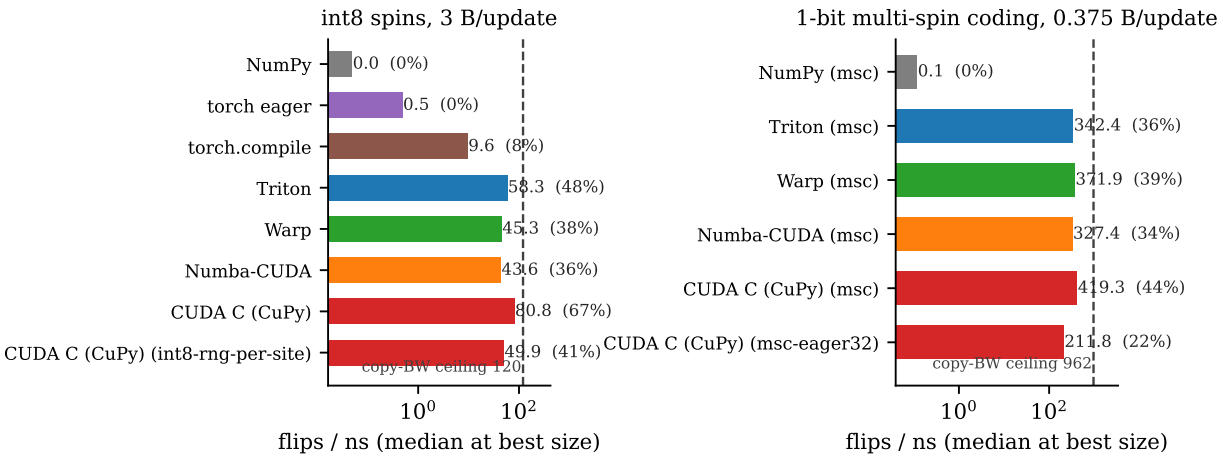


Figure 3: Best throughput of each backend/variant (2D Metropolis at T_c, hot start); labels give flips/ns and the percentage of the copy-bandwidth ceiling under the minimal traffic model. Dashed: copy-bandwidth ceiling.

memory-bound, and the RNG is cheap once one Philox call serves four sites (the per-site ablation runs at 49.9 flips/ns versus 80.8).

Multi-spin coding. Packing cuts the traffic eightfold and raises the memory ceiling to 962 flips/ns, close to the per-warp Philox ceiling of 1144 flips/ns. Random numbers are the largest single cost: the eager ablation that always draws 32 bits per spin reaches 211.8 flips/ns, the lazy kernel 419.3 flips/ns. The best variant overall, CUDA C (CuPy) (msc) at $L = 4096$, reaches 419.3 flips/ns, 43.6 % of the copy ceiling (157.2 GB/s effective, 35.1 % of nominal) and 36.7 % of the per-warp Philox ceiling, 8558× the NumPy reference. Neither ceiling is reached; we have not profiled which on-chip resource limits the packed kernels. The best kernel written entirely in a Python DSL, Warp (msc), reaches 371.9 flips/ns (38.6 % of copy bandwidth).

3D. The simple-cubic int8 kernels reach 77.2 flips/ns (CUDA C (CuPy), $L = 256$, 64.2 % of copy bandwidth).

Comparison with published codes. Romero et al. [16] report 420.8 flips/ns for their optimised multi-spin code on a V100-SXM with HBM2 memory, about twice the nominal bandwidth of our laptop part, using 4-bit spins and one 32-bit uniform per spin; Block et al. [12] report 7.9 flips/ns on a Tesla C1060. Our best kernel reaches 419.3 flips/ns, about the same absolute rate on a part with half the nominal bandwidth. The gain is algorithmic (1 instead of 4 bits per spin, 12.6 instead of 32 random bits per spin), not a property of the host language, and it is a single-device statement on different hardware generations, not a claim to exceed their throughput or their multi-GPU aggregates (but see section 6.1).

6.1 Head-to-head with the reference CUDA implementation

To compare on equal hardware we built and ran NVIDIA's reference code [17] (commit `935796d`) on our GPU. It contains five single-GPU programs for 2D checkerboard Metropolis: a basic CUDA C and a basic Python version (Numba kernels, CuPy cuRAND uniforms), a tensor-core version and

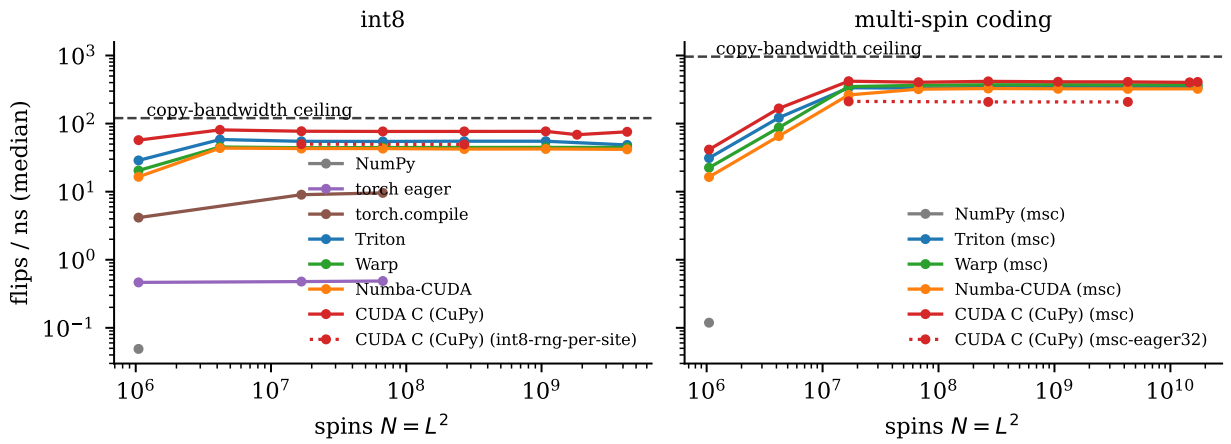


Figure 4: Throughput against lattice size. Small lattices are bound by kernel-launch latency; large lattices reach a plateau below the roofs of fig. 5. Dotted lines are the RNG ablations.

the optimised 4-bit multi-spin code of Romero et al. [16], and the current 1-bit code (`cuIsingModel`) of Bisson et al. [18], which draws its acceptance bits from a 16-entry lookup table (four random bits per 32-bit draw). We compiled them with `nvcc` 13.4 for `sm_86` and did not change their sources or algorithms. The only patches are build flags: the target architecture, C++17 for the CUB shipped with CUDA 13, and a force-included header that restores two runtime-API overloads removed in CUDA 13 (for the tensor-core program it also provides the half-to-double CUB sum used for the final magnetisation, outside the timed loop). The basic Python script runs unchanged, with `cupy.cuda.curand` aliased to its current module. `bench/counterparts.py` builds everything, runs both codes on the same lattices at $T = T_c$, hot start, $J = 1$, periodic boundaries, and records the commit and every patch in `results/counterparts.json`.

Both codes report the same metric: L^2 times the number of sweeps divided by the time, where a sweep updates both colours once. We parse the rate each program prints, so their timed regions stay as written (host clock or CUDA events around the update loop; the basic codes include the cuRAND call that fills one uniform per site). Each configuration runs once to size the sweep count, then five times (median reported). As a check that both simulate the same model, the energy per spin after 200 sweeps from a hot start on $L = 4096$ (three seeds each) is -1.3826 for `cuIsingModel` and -1.3832 for our packed CUDA C kernel, within 0.8 standard errors. The basic CUDA and basic Python programs share the cuRAND stream and give identical magnetisations.
Provisional: the numbers in this subsection come from a short run on a shared GPU (mode `quick` of `bench/counterparts.py`); the final numbers come from its `--full` mode on an idle GPU.

Byte-per-spin codes. Basic CUDA C reaches 21.4 flips/ns ($L = 4096$); our best int8 kernel (CUDA C (CuPy)) reaches 63.1 flips/ns, 2.95× as much. Their program writes a 4-byte uniform per site to DRAM and reads it back; ours draws random numbers inside the update kernel. The basic Python program reaches 3.0 flips/ns; our best Python-DSL int8 kernel (Triton) reaches 46.9 (15.49×). Its Numba kernel computes the acceptance probability in double precision (Python floats are 64-bit; checked in the PTX), which a GeForce GPU executes at 1/64 of the FP32 rate (the V100 of the original paper: 1/2). The slowdown comes from FP64 arithmetic on a consumer GPU, not from Python itself. The tensor-core program reaches 11.1 flips/ns.

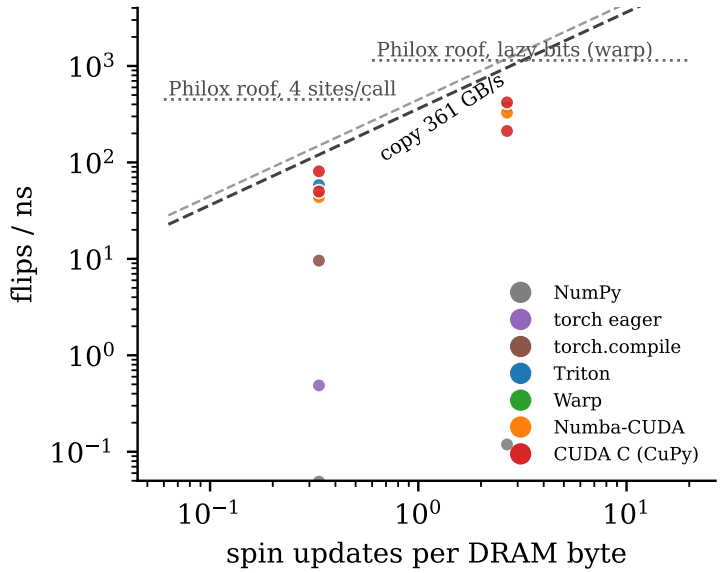


Figure 5: Roofline: throughput against spin updates per DRAM byte. Dashed: measured copy and nominal bandwidth; dotted: Philox throughput divided by calls per update.

Multi-spin codes. The 4-bit code of Romero et al. [16] reaches 169.2 flips/ns ($L = 4096$, 70.3 % of copy bandwidth at 1.5 B/update), and the 1-bit `cuIsingModel` reaches 201.3 flips/ns ($L = 4096$, 20.9 % of copy bandwidth). On this GPU our best packed kernel (CUDA C (CuPy), 364.8 flips/ns at $L = 4096$) is faster than the 1-bit reference (ratio 1.81) and faster than the 4-bit one (ratio 2.16). The best kernel written in a Python DSL (Warp, 300.8 flips/ns) is faster than the 1-bit reference (ratio 1.49). None of the packed codes reaches its memory ceiling, so the difference lies in the work per spin. The 4-bit code moves four times the bytes and draws one 32-bit uniform per spin. `cuIsingModel` draws two 4-bit table lookups per four spins (16 random bits per spin) at every temperature. Our lazy comparison draws 12.6 bits per spin, counted at the slowest thread of each warp at T_c. `cuIsingModel` also synchronises with the host after every half-sweep, which costs most on small lattices. We have not profiled their kernels on this GPU and do not attribute the whole gap to the random numbers; their kernel parameters were tuned on other GPUs, so the ranking holds for this laptop GPU only and says nothing about their multi-GPU results.

7 Discussion

What carries the speed. Three choices matter more than the language: a layout that touches only active spins, one random draw per spin amortised over many spins (four sites per Philox call for int8; 64 lanes per random word for packed spins), and a lazy comparison that spends random bits only on undecided lanes. Once these are in place, kernels written in Python reach 72 % (int8, Triton) and 89 % (packed, Warp) of the throughput of the NVRTC-compiled CUDA C reference; none of them overtakes it.

Threats to validity. (i) A single laptop GPU: clocks and power limits throttle under sustained load, and the copy ceiling itself varies by a few percent between runs; we report medians with min/max. (ii) During development the GPU was shared; every number reported here comes

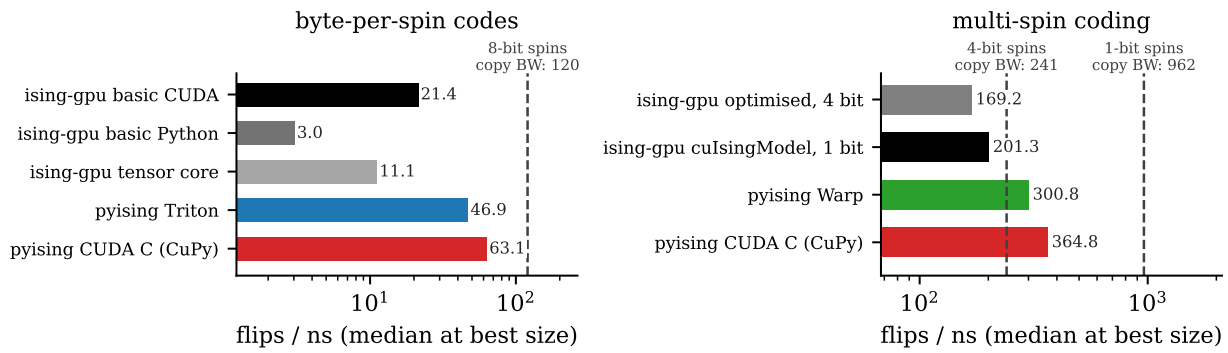


Figure 6: Same-GPU head-to-head, 2D Metropolis at T_c: best median over lattice sizes of the NVIDIA/ising-gpu programs (black, grey) and of pyising (colours). Dashed: copy-bandwidth ceilings for 8-, 4- and 1-bit spins under the minimal traffic model.

from one serial run on an idle GPU. (iii) The CUDA C reference is compiled by NVRTC with `-use_fast_math`, not by `nvcc` with tuned flags; a different compiler could change the reference by some percent. (iv) The lazy kernel's cost depends on temperature (fewer undecided lanes deep in the ordered phase); we benchmark at T_c. (v) The packed variant is implemented for 2D Metropolis only; heat bath and 3D use int8 spins.

8 Conclusion

A single Python package drives seven backends of the same algorithm with bit-identical results and validated physics. On a laptop GPU, Python-hosted multi-spin-coding kernels reach 419.3 spin flips per ns (43.6 % of the memory roofline; 371.9 for a kernel written entirely in a Python DSL) by spending effort where the roofline says it matters: bytes per update and random bits per spin.

Code availability

Code, benchmark JSON, figure scripts and this paper: `https://github.com/walidhemati/pyising` (MIT licence). `bench/run.py --full`, `bench/counterparts.py --full`, `figures/make_figures.py` and `tectonic paper/paper.tex` regenerate every number and figure.

References

[1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[2] Ulli Wolff. Collective Monte Carlo updating for spin systems. *Physical Review Letters*, 62(4): 361–364, 1989. doi: 10.1103/PhysRevLett.62.361.

[3] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance*

Computing, Networking, Storage and Analysis (SC '11), pages 1–12, New York, NY, USA, 2011. ACM. doi: 10.1145/2063384.2063405.

[4] Lars Onsager. Crystal statistics. I. a two-dimensional model with an order-disorder transition. *Physical Review*, 65(3–4):117–149, 1944. doi: 10.1103/PhysRev.65.117.

[5] C. N. Yang. The spontaneous magnetization of a two-dimensional Ising model. *Physical Review*, 85(5):808–816, 1952. doi: 10.1103/PhysRev.85.808.

[6] Arthur E. Ferdinand and Michael E. Fisher. Bounded and inhomogeneous Ising models. I. specific-heat anomaly of a finite lattice. *Physical Review*, 185(2):832–846, 1969. doi: 10.1103/PhysRev.185.832.

[7] Alan M. Ferrenberg, Jiahao Xu, and David P. Landau. Pushing the limits of Monte Carlo simulations for the three-dimensional Ising model. *Physical Review E*, 97(4):043301, 2018. doi: 10.1103/PhysRevE.97.043301.

[8] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114.

[9] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in Monte Carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987. doi: 10.1103/PhysRevLett.58.86.

[10] K. Binder. Finite size scaling analysis of Ising model block distribution functions. *Zeitschrift für Physik B Condensed Matter*, 43(2):119–140, 1981. doi: 10.1007/BF01293604.

[11] Tobias Preis, Peter Virnau, Wolfgang Paul, and Johannes J. Schneider. GPU accelerated Monte Carlo simulation of the 2D and 3D Ising model. *Journal of Computational Physics*, 228 (12):4468–4477, 2009. doi: 10.1016/j.jcp.2009.03.018.

[12] Benjamin Block, Peter Virnau, and Tobias Preis. Multi-GPU accelerated multi-spin Monte Carlo simulations of the 2D Ising model. *Computer Physics Communications*, 181(9):1549–1556, 2010. doi: 10.1016/j.cpc.2010.05.005.

[13] Martin Weigel. Simulating spin models on GPU. *Computer Physics Communications*, 182(9): 1833–1836, 2011. doi: 10.1016/j.cpc.2010.10.031.

[14] Martin Weigel. Performance potential for simulating spin models on GPU. *Journal of Computational Physics*, 231(8):3064–3082, 2012. doi: 10.1016/j.jcp.2011.12.008.

[15] Kun Yang, Yi-Fan Chen, Georgios Roumpos, Chris Colby, and John Anderson. High performance Monte Carlo simulation of Ising model on TPU clusters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*, pages 1–15. ACM, 2019. doi: 10.1145/3295500.3356149.

[16] Joshua Romero, Mauro Bisson, Massimiliano Fatica, and Massimo Bernaschi. High performance implementations of the 2D Ising model on GPUs. *Computer Physics Communications*, 256:107473, 2020. doi: 10.1016/j.cpc.2020.107473.

[17] NVIDIA. ising-gpu: GPU-accelerated Monte Carlo simulations of 2D Ising model. https://github.com/NVIDIA/ising-gpu, 2025. Commit 935796d.

[18] Mauro Bisson, Massimo Bernaschi, Massimiliano Fatica, Nikolaos G. Fytas, Isidoro González-Adalid Pemartín, Víctor Martín-Mayor, and Alexandros Vasilopoulos. Massive-scale simulations of 2D Ising and Blume-Capel models on rack-scale multi-GPU systems. *Computer Physics Communications*, 315:109690, 2025. doi: 10.1016/j.cpc.2025.109690.